

Tutorial - Semana 7, Encontro 2

Introdução

O Encontro 1 fechou o último sistema novo do Módulo 2: `SaveData`, `SaveComponent` e `Checkpoint`, somados a tudo que já existia desde a Semana 4 (`GameManager`, `SaveManager`, contrato `Interactable`, `Signals`, `ItemData` /Enum). Este encontro não introduz nenhum sistema novo — ele existe para verificar que todos esses sistemas, construídos em semanas separadas, realmente se conectam em um único fluxo jogável, sem lógica duplicada entre eles. É também o primeiro momento formal da disciplina em que cada grupo precisa justificar, para o professor e para os colegas, por que fez cada escolha de arquitetura — habilidade cobrada de forma crescente até a Semana 17.

Objetivos da semana

- Integrar `GameManager`, `SaveManager`, contrato `Interactable`, `Signals`, `ItemData` /Enum e `SaveData` / `Checkpoint` em um único fluxo jogável.
- Justificar, em termos de arquitetura, as escolhas de integração adotadas pelo grupo.
- Participar do Code Review e do Playtest coletivo de encerramento da Unidade II.

Resultado esperado ao final da semana

Este tutorial cobre apenas o **Encontro 2**: ao final dele, cada grupo tem um fluxo jogável único — portas, alavancas, `Pickup`, `Chest` e `Checkpoint` conectados entre si — com progresso real persistido entre sessões, apresentado em Code Review e testado pelos colegas no Playtest coletivo. Isso encerra a Unidade II (Construir Sistemas).

Pré-requisitos

- `SaveData`, `SaveComponent`, ao menos um `Pickup`, um `Chest` e um `Checkpoint` funcionais, e o ciclo de spawn fechado (`PlayerStart` + `GameManager.spawn_player()` escolhendo o checkpoint ativo), do Encontro 1 desta semana.
 - `GameManager`, `SaveManager`, contrato `Interactable`, `Signals` (Semanas 4 e 5) e `ItemData` /Enum com o conjunto de itens do grupo (Semana 6), todos sem alterações pendentes.
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto completo do grupo até o Encontro 1 desta semana, com todos os sistemas do Módulo 2 presentes, mesmo que ainda não conectados entre si em um único fluxo.

Arquivos necessários

- Nenhum arquivo externo adicional.

Assets utilizados

- Os mesmos assets do Mini Dungeon já em uso pelo grupo desde a Semana 5, importados em `assets/dungeon/` desde a Semana 1.

Projeto esperado

- Projeto aberto no Godot 4.7, com portas, alavancas, `Pickup`, `Chest` e `Checkpoint` existindo no nível, ainda que dispersos ou não totalmente conectados entre si.

Imagem sugerida

Objetivo: ilustrar o fluxo jogável integrado que o grupo deve alcançar ao final do encontro.

Enquadramento: mapa simplificado do nível de teste do grupo, visto de cima. Elementos

importantes: ícones de porta, alavanca, baú e checkpoint distribuídos no espaço, conectados por uma linha pontilhada representando o caminho jogável do início ao objetivo. Destaque: a linha de fluxo contínua, mostrando que os sistemas não são ilhas isoladas. Legenda sugerida: "Fluxo jogável integrado: cada sistema do Módulo 2 conectado ao caminho único do jogador."

Parte 1 — Integração: conectando os sistemas do módulo em um único fluxo

Objetivo

Conectar todos os objetos interativos do grupo (portas, alavancas, baús, `Checkpoint`) em um único fluxo jogável de ponta a ponta.

Conceito

Um Vertical Slice não é a soma isolada de sistemas testados cada um em seu próprio canto — é a integração deles em uma experiência única. Até aqui, cada sistema do Módulo 2 foi construído e testado separadamente: uma porta na Semana 5, o `ItemData` na Semana 6, um `Pickup`, um `Chest` e um `Checkpoint` no Encontro 1 desta semana. A pergunta deste encontro é diferente de "cada sistema funciona sozinho?" — é "o jogador consegue percorrer o nível do início ao fim, interagindo com todos eles em sequência, sem que nenhum sistema precise ser retrabalhado para se encaixar nos demais?". Se o contrato `Interactable` e os Signals foram bem desenhados desde a Semana 5, a resposta deve ser sim, sem exigir nenhuma alteração estrutural agora — apenas posicionamento e conexão.

Passo a passo

1. Abra o nível de teste principal do grupo (zona externa ou estrutura interna, conforme o progresso do grupo).
2. Posicione (ou reposicione) as instâncias de `Door`, `Lever`, `Pickup`, `Chest` e `Checkpoint` já existentes, formando um caminho único que o jogador percorre do ponto inicial até um ponto final.
3. Confirme que uma alavanca (`Lever`) efetivamente controla a abertura de uma porta (`Door`) próxima, reutilizando o Signal já conectado desde a Semana 5.
4. Confirme que ao menos um `Pickup` e um `Chest` no caminho concedem um item do conjunto `ItemData` do grupo (Semana 6), que o Signal `item_collected` de cada um está conectado ao handler único, e que o `nome` do item aparece em `SaveManager.itens_coletados` sem duplicatas.
5. Posicione o `Checkpoint` do Encontro 1 em um ponto lógico do caminho (por exemplo, após a primeira sala resolvida), de forma que alcançá-lo grave o progresso já obtido até ali.
6. Percorra o caminho completo do início ao fim, interagindo com cada objeto na ordem esperada, sem editar código durante o percurso.

7. Feche o jogo, reabra, e confirme que (a) o progresso salvo no `Checkpoint` (itens coletados até ali) é recuperado ao carregar o `SaveData` e (b) o Player nasce na posição do último `Checkpoint` alcançado, e não no `PlayerStart` — o ciclo de spawn fechado na Parte 5 do Encontro 1.

Resultado esperado

O grupo percorre o nível do início ao fim interagindo com portas, alavancas, baús e o `Checkpoint` em um único fluxo contínuo, sem nenhum sistema precisar ser alterado estruturalmente para se conectar aos demais.

Verificando

1. Confirme que nenhum objeto interativo do grupo duplica lógica já existente em outro (por exemplo, duas implementações diferentes de detecção de proximidade).
2. Confirme que o progresso persistido pelo `Checkpoint` reflete corretamente os itens coletados até aquele ponto do fluxo, e não o estado completo do nível.
3. Peça a um colega de outro grupo (fora do horário formal de Playtest, se houver tempo) para percorrer o fluxo sem instruções verbais, e observe se ele completa o caminho sem travar.

Problemas comuns

- Descobrir, ao tentar integrar, que um sistema anterior (por exemplo, a porta da Semana 5) foi implementado de forma isolada e precisa de retrabalho para se conectar aos demais: tratar isso como sinal de que o contrato `Interactable` não foi seguido à risca, e usar o tempo de laboratório para corrigir a conexão, não para reconstruir o sistema do zero.
- Conectar sistemas de forma funcional, mas sem nenhuma lógica clara de por que aquele objeto está naquele ponto do fluxo: reforçar que a integração deve fazer sentido como progressão, não apenas como demonstração técnica de que os sistemas "encostam" uns nos outros.
- Deixar o `Checkpoint` gravando o estado completo do `GameManager` em vez de apenas os dados relevantes definidos em `SaveData`: revisar o Encontro 1 se isso ocorrer.

Boas práticas

- Preferir conectar sistemas existentes a criar novos objetos interativos neste encontro — o objetivo é integração, não expansão de escopo.
- Testar o fluxo completo do começo ao fim pelo menos duas vezes antes do Code Review, incluindo um teste após fechar e reabrir o jogo.

Comparação com Unity

Nenhuma comparação nova é introduzida neste encontro — é o momento de consolidar as comparações já feitas nas Semanas 4 a 7: `GameManager` (Autoload) versus a ausência de um equivalente formal direto na Unity; contrato `Interactable` (duck typing/interface do Orchestrator) versus Interfaces em C#; Signals versus UnityEvent/C# Actions; `ItemData` /Enum versus `ScriptableObject` / enum ; `SaveData` /FileAccess versus `PlayerPrefs` /JSON. O ponto central é que, em ambas as engines, um bom desacoplamento entre sistemas é o que torna a integração final simples — o problema de "juntar tudo no final" só aparece quando o desacoplamento foi malfeito desde o início.

Parte 2 — Preparando a justificativa de arquitetura para o Code Review

Objetivo

Preparar, em grupo, a justificativa das escolhas de arquitetura adotadas ao longo do Módulo 2.

Conceito

Até aqui, a disciplina cobrou principalmente que cada sistema funcionasse. A partir deste Code Review, passa a cobrar também que o grupo explique por quê: por que aquele Enum de categoria e não outro; por que aquele ponto específico foi escolhido para o `Checkpoint` ; por que a alavanca aciona a porta via Signal em vez de uma referência direta. Essa exigência não é burocracia — é o mesmo tipo de pergunta que qualquer revisor técnico faz em uma equipe de desenvolvimento real, e a disciplina introduz essa prática de forma crescente a partir de agora.

Passo a passo

1. Em grupo, revisem juntos cada sistema construído no Módulo 2: `GameManager` , `SaveManager` , contrato `Interactable` , Signals, `ItemData` /Enum, `Pickup` / `Chest` + handler de coleta, `SaveData` / `SaveComponent` , `Checkpoint` .
2. Para cada sistema, escrevam (em papel ou em um documento curto) uma frase respondendo "por que fizemos dessa forma, e não de outra forma possível?".
3. Revisem a Rubrica 4 (Code Review) do Sistema de Avaliação com o professor ou material de apoio, focando em nomenclatura, modularidade e reutilização do contrato `Interactable` sem

lógica duplicada.

4. Escolham um integrante do grupo para conduzir a apresentação e outro para responder perguntas técnicas específicas, evitando que a apresentação toda recaia sobre uma única pessoa.
5. Ensaaiem uma apresentação de fluxo (não de código isolado): mostrar o jogo sendo jogado do início ao fim, pausando brevemente em cada sistema para explicar a decisão de arquitetura por trás dele.

Resultado esperado

Cada grupo consegue apresentar o fluxo jogável completo ao professor, justificando com as próprias palavras por que cada escolha de arquitetura foi feita, sem depender de um roteiro decorado.

Verificando

1. Peça a um integrante do grupo que não seja o "porta-voz" principal que explique, sozinho, por que o `Checkpoint` foi posicionado onde está.
2. Confirme que o grupo consegue diferenciar, em suas próprias palavras, `SaveManager` (entre cenas) de `SaveData` (entre sessões) sem consultar anotações.

Problemas comuns

- Preparar uma apresentação que descreve cada sistema isoladamente ("aqui está nosso GameManager, aqui está nosso SaveData") em vez de demonstrar o fluxo integrado: reforçar que o Code Review avalia a integração, não a soma de partes.
- Não conseguir justificar uma escolha além de "foi a primeira ideia que tivemos": usar a pergunta "por que esse caminho e não outro possível?" durante o ensaio, para chegar preparado à pergunta real do professor.

Boas práticas

- Ensaiar a apresentação rodando o jogo de verdade, não descrevendo o código em abstrato — o Code Review desta disciplina é sobre o sistema em funcionamento, não sobre slides.
- Distribuir a explicação entre os integrantes do grupo, evitando que apenas uma pessoa saiba justificar as decisões técnicas.

Comparação com Unity

Não aplicável nesta parte — é um momento de preparação de apresentação, não de implementação técnica.

Parte 3 — Code Review e Playtest coletivo

Objetivo

Apresentar o fluxo jogável integrado ao professor (Code Review) e testá-lo com os colegas de outros grupos (Playtest coletivo).

Conceito

O Code Review e o Playtest avaliam ângulos diferentes e complementares do mesmo entregável. O Code Review, conduzido pelo professor a partir da Rubrica 4, observa a arquitetura por trás do fluxo: nomenclatura, modularidade, reutilização do contrato `Interactable` sem lógica duplicada entre `Player` e os interativos. O Playtest coletivo, conduzido pelos colegas, observa o resultado do ponto de vista de quem joga sem conhecer a implementação: o fluxo funciona, é compreensível, e não trava em nenhum ponto? Um projeto tecnicamente bem arquitetado ainda pode falhar no Playtest se a experiência não for clara para quem joga pela primeira vez — e vice-versa.

Passo a passo

1. Quando chamado pelo professor, execute o projeto do início e demonstre o fluxo completo, pausando nos pontos combinados na Parte 2 para justificar as escolhas de arquitetura.
2. Responda às perguntas do professor durante o Code Review, seguindo o roteiro da Rubrica 4 do Sistema de Avaliação.
3. Durante o Playtest coletivo, disponibilize o projeto do grupo para que colegas de outro grupo joguem sem instruções prévias detalhadas.
4. Observe (sem intervir, salvo travamentos técnicos) onde o jogador convidado hesita, erra o caminho, ou não entende o que fazer.
5. Anote rapidamente os pontos observados, mesmo que não haja tempo de corrigi-los nesta semana — servirão de referência para o Módulo 3, quando o HUD (Semana 9) passar a comunicar objetivo e estado de jogo de forma mais explícita.
6. Realize o mesmo processo como jogador no projeto de outro grupo, dando feedback construtivo e específico.

Resultado esperado

Cada grupo passa pelo Code Review demonstrando e justificando o fluxo integrado, e pelo Playtest coletivo testando e sendo testado por outro grupo, encerrando formalmente a Unidade II.

Verificando

1. Confirme que o Code Review cobriu, ao menos brevemente, cada um dos sistemas do Módulo 2 — nenhum sistema apresentado isoladamente sem relação com o fluxo.
2. Confirme que o Playtest coletivo gerou ao menos um ponto de observação anotado pelo grupo, mesmo que positivo.

Problemas comuns

- Tratar o Playtest como uma demonstração guiada pelo próprio grupo em vez de deixar o colega jogar livremente: reforçar que o valor do Playtest está justamente em observar alguém sem o conhecimento prévio do fluxo.
- Ficar na defensiva diante de feedback do Playtest ou de perguntas do Code Review: reforçar que o objetivo desta semana é identificar pontos de melhoria para o restante do semestre, não obter uma nota perfeita imediata.

Boas práticas

- Registrar os pontos observados no Playtest de forma objetiva, para retomar quando o HUD (Semana 9) e a interação ampliada (Semana 10) estiverem disponíveis.
- Tratar o feedback dos colegas com o mesmo profissionalismo esperado em uma equipe real de desenvolvimento.

Comparação com Unity

Não aplicável nesta parte — Code Review e Playtest são práticas de processo, não recursos específicos de uma engine.

Ao final da semana

Ao final da Semana 7, cada grupo possui um Vertical Slice com `GameManager` / `SaveManager` (Autoload), contrato `Interactable`, Signals, `ItemData` / Enum e `SaveData` / `Checkpoint` integrados em um único fluxo jogável, com progresso real persistido entre sessões — não apenas entre cenas. O grupo passou pelo Code Review (Rubrica 4) e pelo Playtest coletivo (rubrica correspondente do Sistema de Avaliação), encerrando a Unidade II — Construir Sistemas.

Segundo o `PROJECT_ARCHITECTURE.md` (seção 6), este resultado corresponde ao "Produto do Módulo 2: gameplay funcional, com portas, baús, alavancas, checkpoints e progresso persistente integrados em um único fluxo" — base direta do Módulo 3, que inicia com o `HealthComponent` (Semana 8), reutilizando o mesmo padrão de Component já estabelecido pelo `SaveComponent`.

Desafio

Não há desafio de solução livre neste encontro: a entrega da semana é a própria apresentação da integração completa do Módulo 2, avaliada por Code Review e Playtest coletivo. **Entrega: gameplay funcional consolidado do Módulo 2.**

Checklist

- ☐ Portas, alavancas, `Pickup`, `Chest` e `Checkpoint` conectados em um único fluxo jogável, do início ao fim
- ☐ Progresso persistido pelo `Checkpoint` confirmado após fechar e reabrir o jogo
- ☐ Nenhum sistema do Módulo 2 retrabalhado desnecessariamente durante a integração
- ☐ Justificativa de arquitetura preparada para cada sistema do módulo
- ☐ Code Review realizado com o professor (Rubrica 4)
- ☐ Playtest coletivo realizado com outro grupo, com pontos de observação anotados

Glossário

- **Fluxo jogável integrado:** sequência única e contínua de interações que o jogador percorre, conectando todos os sistemas construídos, em oposição a sistemas testados isoladamente.
- **Code Review:** avaliação técnica da arquitetura do projeto, conduzida pelo professor a partir da Rubrica 4 do Sistema de Avaliação.

- **Playtest coletivo:** teste do fluxo jogável conduzido por colegas de outro grupo, sem conhecimento prévio da implementação.

Referências

- Godot Documentation — Saving Games:
https://docs.godotengine.org/en/stable/tutorials/io/saving_games.html
- Godot Documentation — Resources:
<https://docs.godotengine.org/en/stable/tutorials/scripting/resources.html>
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa) — PlayerPrefs: <https://docs.unity3d.com/Manual/class-PlayerPrefs.html>
- Canais recomendados para consulta complementar (não substituem a documentação oficial):
GDQuest (<https://www.youtube.com/@GDQuest>), Clear Code
(<https://www.youtube.com/@ClearCode>)